

Eid al-Adha Livestock Price Forecast



INSIGHTS AT EVERY STEP

Done by: Ali Y Hussein

[GitHub Repo](#)

1. Project overview

This project is an end-to-end data science and MLOps exercise that forecasts weekly sheep and goat meat prices around the time of Eid al-Adha. The goal is to build something that looks and feels like a real data product: a reproducible pipeline, a trained model, and a deployed Streamlit dashboard where users can explore historical prices and short-term forecasts.

The data comes from the official **EU Agri-food Data Portal** sheep and goat meat prices API, which provides weekly prices for heavy and light lamb across EU member states. On top of that, the project engineers features related to the timing of Eid al-Adha so that you can analyze how prices behave around the holiday.^[1]

2. Objectives

The project has three main objectives:

1. Data engineering:

- Extract weekly lamb prices from a real API endpoint.
- Clean and transform the data into a modeling-ready dataset with time and holiday features.

2. Forecasting model:

- Train a regression model that predicts future prices based on lagged prices, calendar variables, and simple Eid-related features.
- Persist the model so that it can be reused by the application or retrained when needed.

3. Interactive application:

- Provide a **Streamlit** dashboard where a user can choose a country and lamb category and immediately see historical trends plus an 8-week forecast.

The result is something you can show in a portfolio as a concrete example of: data collection, feature engineering, modeling, and deployment.

3. Data source and preprocessing

3.1 Data source

The raw data is EU-wide sheep and goat meat prices retrieved via the Agrifood API (`/api/sheepAndGoat/prices`). Each record contains:

- Member state code and name.
- Begin and end dates of the week.

- Price value and unit (usually per 100 kg).
- Week number, marketing year, market name, and category (Heavy or Light lamb).^[1]

These API calls were performed once during dataset construction, and the results were exported into a static CSV file:

- `data/sheep_goat_prices_eu.csv` – canonical dataset used throughout the project.

This design (static CSV checked into the repo) makes the project reproducible without requiring live API access in the deployed app.

3.2 Data preparation

All data preparation is centralized in `src/data_prep.py`.

Key steps:

1. Loading:

- `load_raw()` reads the CSV from `data/`.

2. Cleaning prices:

- The original `price` field is a string like "€531.00".
- The script strips the currency symbol and commas and converts it to a float `price_numeric` (still per 100 kg).

3. Handling dates and time:

- `beginDate` and `endDate` are parsed as `datetime`.
- A representative `week_date` is set to the midpoint between `beginDate` and `endDate`.
- `year` and `week` are derived from `week_date` using ISO week numbers.

4. Filtering:

- Very old data is dropped by keeping only years from a configurable minimum (e.g., 2015 onward).

This produces a clean, long time series per country and category that is ready for feature engineering.

4. Feature engineering

Feature creation is handled in `src/features.py`.

4.1 Calendar and Eid features

The module uses a dictionary of known **Eid al-Adha dates** per Gregorian year (2015–2025) stored in `src/config.py`.

From that, the following are added:

- `month`: Month of `week_date`.
- `dayofyear`: Day of year of `week_date`.
- `is_eid_year`: Binary flag indicating whether the year has an Eid date defined.
- `days_to_eid`: Integer difference between `week_date` and the Eid date for that year (if available).
- `eid_season`: Boolean indicating whether the week falls in a window around Eid (for example from 7 days before to 21 days after).

These features allow the model and analysis to reason about how close a given week is to Eid, capturing seasonal patterns.

4.2 Lag features

To give the model a sense of recent price history, lagged versions of the target are added per country and category:

- `price_numeric_lag_1` – previous week’s price.
- `price_numeric_lag_2` – price two weeks ago.
- `price_numeric_lag_4` – price four weeks ago.

The function `add_lag_features()` groups by `memberStateCode` and `category` and computes these lagged values in time order. Rows with missing lags (at the start of each series) are dropped before training.

5. Modeling

All modeling logic is in `src/modeling.py`.

5.1 Dataset assembly

`build_dataset()` composes the full pipeline:

1. `load_raw()` → load CSV.
2. `clean_and_parse_dates()` → parse dates and numeric prices.
3. `filter_reasonable_years()` → keep recent years.
4. `add_calendar_features()` → month, dayofyear, days_to_eid, etc.
5. `add_lag_features()` → lagged target variables.
6. Drop rows with missing lag values.

5.2 Features and target

The target is `price_numeric` (price per 100 kg). The feature set is split into:

- **Numeric features:**
 - `week`, `month`, `dayofyear`, `days_to_eid`,
 - `price_numeric_lag_1`, `price_numeric_lag_2`, `price_numeric_lag_4`.
- **Categorical features:**
 - `memberStateCode`, `category`.

`get_feature_target()` constructs `X` and `y` and returns them along with the feature name lists.

5.3 Model and pipeline

The model is a **RandomForestRegressor** wrapped in a scikit-learn `Pipeline` with a `ColumnTransformer` for preprocessing:

- Numeric pipeline:
 - `SimpleImputer(strategy='median')` for missing values (mainly `days_to_eid` for years outside the Eid mapping).
 - `StandardScaler` to normalize scales.
- Categorical pipeline:
 - `OneHotEncoder(handle_unknown='ignore')` to encode country and category.
- Core model:

- `RandomForestRegressor(n_estimators=300, random_state=42, n_jobs=-1)`.

The script splits data into train and test sets, trains the pipeline, and then prints MAE and RMSE so that you have a quick sense of prediction quality. Finally, the trained pipeline is serialized to `models/rf_price_forecast.joblib`.

On Streamlit Cloud, the first run trains the model automatically if the `.joblib` file is missing; subsequent runs reuse the saved model.

6. Application architecture

The app is built with **Streamlit** and lives in `app/app.py`.

6.1 High-level flow

1. Path setup:

- Early in the script, `sys.path` is updated so that `src` can be imported when running from the `app` directory (important for both local and deployed environments).

2. Data loading (cached):

- `load_dataset()` uses the same data prep and feature functions as modeling, but only keeps rows with valid lag values.
- It is wrapped in `@st.cache_data` to avoid recomputing on every interaction.

3. Model loading (cached):

- `load_model()` checks if the trained model file exists; if not, it calls `train_model()` from `src.modeling`, then loads it with `joblib.load`.
- Wrapped in `@st.cache_resource` so the model is instantiated only once per session.

4. User interface:

- Sidebar filters:
 - `Country` (`memberStateCode`).
 - `Category` (Heavy or Light lamb; often filtered to categories available for the selected country).
- Main page:

- Title: “Eid al-Adha Livestock Price Forecast”.
- Historical prices chart for the selected combination.
- Forecast for the next 8 weeks.

6.2 Forecast logic in the app

The forecasting section:

1. Filters the dataset to the chosen country/category.
2. Sorts by `week_date`.
3. Takes the last row as the base state for future predictions.
4. Iteratively moves the week forward by 1 to 8 weeks, updating:
 - `week_date`, `week`, `month`, `dayofyear`, `days_to_eid`.
 - Uses the final available lag values from the last observed row (this could be refined further).
5. For each future row, constructs the same feature vector expected by the model and calls `model.predict()`.
6. Displays the forecasted prices as a line chart.

If the filtered data is empty (e.g., a country has no Light Lamb data), the app shows a friendly warning instead of crashing.

7. Repository layout

The GitHub repository is organized to be familiar to anyone used to Python data projects.^[2]

- `app/`
 - `app.py` – Streamlit UI and forecasting logic.
- `src/`
 - `config.py` – configuration (data path, model directory, Eid dates).
 - `data_prep.py` – loading and cleaning the raw CSV.
 - `features.py` – feature engineering utilities.
 - `modeling.py` – training pipeline and model persistence.

- `evaluation.py` – quick dataset descriptive statistics and sanity checks.
- `data/`
 - `sheep_goat_prices_eu.csv` – cleaned export of EU weekly lamb prices.
- `models/`
 - (Empty in the repo; model file is created at runtime and ignored by Git.)
- `requirements.txt` – Python dependencies for both local use and deployment.
- `README.md` – high-level overview and quickstart instructions.

This structure is intentional: everything related to domain logic and data is under `src/`, while the UI and view layer is in `app/`, which makes it easier to later plug this model into a different interface if needed.

8. How to run locally

From the project root:

1. Create a virtual environment and install dependencies:

```
python -m venv .venv
# Windows:
.venv\Scripts\activate
pip install -r requirements.txt
```

2. (Optional) Train or retrain the model:

```
python -m src.modeling
```

3. Launch the dashboard:

```
streamlit run app/app.py
```

Then open `http://localhost:8501` in your browser.

9. Limitations and possible improvements

A few realistic limitations you can mention in an interview or report:

1. Geographic mismatch with Eid emphasis:

- The data is EU-focused, while Eid al-Adha demand spikes are strongest in majority-Muslim countries. The project therefore shows a *methodology* rather than a perfect real-world match of geography and holiday.

2. Simplified holiday feature:

- The `days_to_eid` mapping uses a single official date per year and a symmetric window around it. In reality, market behavior might respond differently on the run-up versus the aftermath of Eid.

3. Random forest as a black box:

- Random forests are robust but not the most interpretable. Adding SHAP values or switching to models like gradient boosting with feature importance plots would make the story more insightful.

4. Lag features only:

- The model relies on lagged price and simple calendar features. More sophisticated time-series techniques (Prophet, SARIMA, or transformers) could capture long-term seasonality and structural breaks better.

5. Single model for all countries and categories:

- A single global model is simpler to maintain but may underfit some markets. Training separate models per country or per region could improve accuracy.

10. Future work

If you want to extend the project, here are natural next steps:

- Integrate additional data sources such as feed prices, transport costs, or macro indicators as exogenous features.
- Add more comprehensive Eid date coverage and perhaps model other Islamic holidays.
- Implement a dedicated evaluation report with error metrics per country and category, not just overall MAE/RMSE.
- Add explainability to the dashboard (e.g., feature importance, partial dependence plots).

- Containerize the app with Docker for more portable deployment across different platforms.

1. <https://agridata.ec.europa.eu/extensions/DataPortal/sheep-and-goat-meat.html>